

UNIVERSIDADE DE MOGI DAS CRUZES – UMC
CURSO DE SISTEMAS DE INFORMAÇÃO
DISCIPLINA: APLICATIVOS WEB

DERICK ALVES MASCHIO

CONSTRUCERTA

SISTEMA DE LOGIN SEGURO COM SPRING BOOT,
THYMELEAF E MONGODB ATLAS

Trabalho desenvolvido como atividade prática individual
da disciplina Aplicativos Web, sob orientação dos
professores Alessandro Horas e Leonardo Alvino.

Mogi das Cruzes – SP
2026

DERICK ALVES MASCHIO

CONSTRUCERTA

**SISTEMA DE LOGIN SEGURO COM SPRING BOOT,
THYMELEAF E MONGODB ATLAS**

Documentação técnica apresentada como parte da atividade prática individual “Login Completo” da disciplina Aplicativos Web, do curso de Sistemas de Informação da Universidade de Mogi das Cruzes (UMC), referente à Avaliação Teórica e Atividade Prática M1.

Mogi das Cruzes – SP

2026

SUMÁRIO

1	Introdução	1
1.1	Objetivos	1
2	Arquitetura e estrutura do sistema	1
2.1	Organização de pacotes	1
2.2	Camada de apresentação (Thymeleaf)	2
2.3	Perfis de acesso (roles)	2
3	Segurança: autenticação e autorização	3
3.1	Hash de senha	3
3.2	Autenticação	3
3.3	Autorização por perfil	3
3.4	Validação de dados	3
3.5	Logout	4
4	Integração com o MongoDB Atlas	4
4.1	Conexão segura	4
4.2	Coleções utilizadas	4
4.3	Inicialização do administrador	4
5	Preparação para temas visuais configuráveis	5
6	Principais decisões de design	5
6.1	Domínio genérico e reaproveitável	5
6.2	Separação entre validação de formulário e regra de negócio	5
6.3	Defesa em profundidade	6
6.4	Sessão persistida no MongoDB	6
7	Considerações finais	6
	REFERÊNCIAS	7

1 INTRODUÇÃO

Este documento apresenta a documentação técnica do **ConstruCerta**, sistema de login seguro desenvolvido em **Java 17**, com o framework **Spring Boot 3**, o motor de templates **Thymeleaf** e o banco de dados **MongoDB Atlas**. O projeto foi proposto como atividade prática individual da disciplina Aplicativos Web e tem como objetivo implementar, de forma completa, as etapas de cadastro, autenticação e autorização de usuários, respeitando boas práticas de segurança da informação. O tema escolhido para a camada de apresentação é o de uma loja de **materiais de construção**, com um catalogo simples de produtos.

O sistema foi concebido de forma **genérica e modular**: o núcleo de autenticação (pacotes `security`, `config` e o modelo `User/Role`) não depende do tema “materiais de construção” adotado nas telas, podendo ser reaproveitado em outros projetos apenas alterando os valores do `enum` de perfis e o tema visual, sem alterações na lógica central.

1.1 Objetivos

Os objetivos específicos atendidos por este trabalho são:

1. implementar cadastro, login e logout de usuários, com hash de senha e validação de dados de entrada;
2. controlar o acesso a rotas e páginas de acordo com o perfil do usuário, com no mínimo três perfis distintos;
3. utilizar o Spring Security como mecanismo central de autenticação e autorização, com criptografia de senha;
4. utilizar o MongoDB Atlas para armazenar usuários e sessões da aplicação;
5. construir a interface com Thymeleaf, mantendo o design separado da lógica de negócio e preparado para temas visuais configuráveis.

2 ARQUITETURA E ESTRUTURA DO SISTEMA

O ConstruCerta segue o padrão arquitetural **MVC** (Model-View-Controller), organizado em pacotes por responsabilidade dentro de `com.construcerta`, conforme descrito a seguir.

2.1 Organização de pacotes

- `config` – configurações transversais da aplicação: segurança (`SecurityConfig`), integração com o MongoDB (`MongoConfig`), tema visual (`ThemeProperties`, `GlobalModelAttributes`) e criação do administrador inicial (`DataInitializer`);

- **model** – entidades de domínio persistidas no MongoDB: User (documento da coleção users), o enum Role e Produto (documento da coleção produtos, item do catalogo de materiais);
- **repository** – interfaces Spring Data MongoDB responsáveis pelo acesso ao banco (UserRepository, ProdutoRepository);
- **security** – adaptação do modelo de domínio para o contrato do Spring Security (CustomUserDetails, CustomDetailsService);
- **service** – regras de negócio de cadastro e administração de usuários (UserService) e do catalogo de produtos (ProdutoService), incluindo validações que não dependem da camada web;
- **dto** – objetos de transferência de dados usados pelos formulários (RegisterForm, AdminUserUpdateForm, ProdutoForm), com as anotações de validação (*Bean Validation*);
- **controller** – camada web (Spring MVC), responsável por receber requisições HTTP e selecionar a view Thymeleaf correspondente.

2.2 Camada de apresentação (Thymeleaf)

As páginas HTML ficam em `src/main/resources/templates`, organizadas por área (auth, cliente, vendedor, admin, error) e por fragmentos reutilizáveis (`fragments/head.html`, `fragments/navbar.html`). Nenhum controller contém regra visual: cores e nome exibido vêm de um único ponto (`ThemeProperties`), injetado automaticamente em todas as telas por um `@ControllerAdvice` (`GlobalModelAttributes`). Dessa forma, a lógica de negócio permanece completamente separada do design, permitindo trocar cores, nome do sistema e, no futuro, a folha de estilos, sem tocar em nenhuma classe Java.

2.3 Perfis de acesso (roles)

O sistema define três perfis distintos, representados pelo enum `Role`:

- **CLIENTE** – perfil padrão, atribuído automaticamente a todo usuário que se cadastra pela tela pública de registro; consulta o catalogo de materiais;
- **VENDEDOR** – perfil intermediário, com acesso à área do cliente e a uma área adicional de gestão do catalogo, onde cadastra e atualiza produtos (nome, categoria, unidade de medida, preço e estoque);
- **ADMIN** – perfil administrativo, com acesso total, incluindo o painel de gestão de usuários, onde é possível promover ou revogar perfis e ativar/desativar contas.

Nenhum usuário consegue se autopromover: o cadastro público sempre cria o perfil `CLIENTE`, e apenas um `ADMIN` autenticado pode alterar perfis de outros usuários, por meio do painel administrativo.

3 SEGURANÇA: AUTENTICAÇÃO E AUTORIZAÇÃO

A segurança da aplicação é centralizada no **Spring Security 6**, configurado na classe `SecurityConfig`.

3.1 Hash de senha

Nenhuma senha é armazenada em texto puro. Todas as senhas passam pelo algoritmo **BCrypt** (`BCryptPasswordEncoder`) antes de serem persistidas no MongoDB Atlas, tanto no cadastro público quanto na criação do administrador inicial. O `BCrypt` gera um *salt* aleatório para cada senha, o que impede ataques de comparação direta entre hashes mesmo quando duas contas usam a mesma senha.

3.2 Autenticação

O login é feito por formulário customizado (`formLogin`), aceitando tanto o nome de usuário quanto o e-mail no mesmo campo (`CustomUserDetailsService`). O `DaoAuthenticationProvider` delega a verificação da senha ao mesmo `PasswordEncoder` usado no cadastro, garantindo que a validação seja sempre consistente.

3.3 Autorização por perfil

O controle de acesso é aplicado em duas camadas independentes (defesa em profundidade):

1. **por rota**, em `SecurityConfig`: as rotas `/admin/**`, `/vendedor/**` e `/cliente/**` exigem, respectivamente, o perfil `ADMIN`, os perfis `VENDEDOR/ADMIN` e qualquer usuário autenticado;
2. **por método**, com a anotação `@PreAuthorize` nos métodos sensíveis de `UserService` e `ProdutoService` (como listagem de usuários, alteração de perfis e manutenção do catálogo), que exigem o perfil adequado independentemente de qual rota os chamou.

3.4 Validação de dados

O formulário de cadastro (`RegisterForm`) usa anotações de *Bean Validation* (`@NotBlank`, `@Email`, `@Size`) para garantir nome de usuário, e-mail e senha em formatos válidos antes de qualquer persistência. O formulário de produto (`ProdutoForm`) também valida campos obrigatórios e valores numéricos não negativos (preço e estoque). Além disso, a camada de serviço impede o cadastro de nomes de usuário e e-mails duplicados, utilizando índices únicos no próprio MongoDB Atlas como segunda barreira de integridade.

3.5 Logout

O logout (/logout) invalida a sessão HTTP corrente e remove os cookies de sessão (JSESSIONID, SESSION), redirecionando o usuário para a tela de login com uma mensagem de confirmação.

4 INTEGRAÇÃO COM O MONGODB ATLAS

O MongoDB Atlas é utilizado como único mecanismo de persistência da aplicação, por meio do **Spring Data MongoDB**.

4.1 Conexão segura

A *connection string* do cluster Atlas nunca é gravada no código-fonte nem versionada no repositório. Ela é lida em tempo de execução pela propriedade `spring.data.mongodb.uri`, que por sua vez é alimentada pela variável de ambiente `MONGODB_URI`:

```
spring.data.mongodb.uri=${MONGODB_URI:mongodb://localhost:27017/construcerta}
```

O arquivo `.gitignore` do projeto exclui explicitamente arquivos que poderiam conter credenciais locais (`.env`, `application-local.properties`), e o passo a passo de configuração do cluster, usuário de banco e regras de rede do Atlas está documentado no `README.md` do repositório.

4.2 Coleções utilizadas

- **users** – armazena os documentos User (usuário, e-mail, hash de senha, nome completo, conjunto de perfis e status de ativação), com índices únicos em `username` e `email`;
- **produtos** – armazena os documentos Produto do catalogo de materiais de construção (nome, categoria, unidade de medida, preço, estoque e status ativo);
- **sessions** – armazena as sessões HTTP autenticadas, por meio do `spring-session-data-mongodb`. Isso garante que a sessão do usuário sobreviva a um reinício da aplicação e possa, no futuro, ser compartilhada entre múltiplas instâncias do sistema.

4.3 Inicialização do administrador

No primeiro boot da aplicação, a classe `DataInitializer` verifica, via `UserRepository`, se já existe algum usuário com perfil ADMIN no Atlas. Caso não exista, um administrador inicial é criado automaticamente, com credenciais também lidas de variáveis de ambiente (`ADMIN_USERNAME`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`). Essa medida evita que o sistema fique sem nenhum usuário capaz de administrar perfis logo depois do primeiro *deploy*.

5 PREPARAÇÃO PARA TEMAS VISUAIS CONFIGURÁVEIS

Um dos requisitos da atividade é que o sistema esteja preparado para receber temas visuais personalizados no futuro, sem exigir alterações estruturais. Essa necessidade foi atendida da seguinte forma:

1. as cores e o nome exibido do sistema são definidos como propriedades de configuração (`app.theme.*`) em `application.properties`, mapeadas para a classe `ThemeProperties`;
2. essas propriedades são injetadas automaticamente em todas as páginas pelo `GlobalModelAttributes` um `@ControllerAdvice` que adiciona o atributo `theme` ao modelo de toda requisição, sem que nenhum controller precise fazer isso manualmente;
3. o fragmento `fragments/head.html` converte essas propriedades em variáveis CSS (`--color-primary`, `--color-secondary` etc.), e a folha de estilos `theme.css` consome exclusivamente essas variáveis.

Como consequência direta dessa arquitetura, é possível aplicar um novo tema visual completo apenas alterando valores de configuração (ou variáveis de ambiente equivalentes), sem recompilar HTML, CSS ou qualquer classe Java. O tema padrão implementado (“ConstruCerta Laranja”) usa uma paleta em tons de laranja/terracota e verde, com um padrão sutil de tijolos ao fundo, gerado inteiramente em SVG embutido no próprio CSS, sem dependência de imagens externas.

6 PRINCIPAIS DECISÕES DE DESIGN

6.1 Domínio genérico e reaproveitável

A entidade `User` e as classes de segurança foram desenhadas sem nenhum campo ou regra específica do tema “materiais de construção”. Essa decisão permite que o mesmo núcleo de login seja reaproveitado em outro projeto futuro apenas trocando os valores do enum `Role` e ajustando as regras de rota correspondentes em `SecurityConfig`, sem reescrever cadastro, login, logout, hash de senha ou persistência de sessão. O catálogo de produtos, por sua vez, fica isolado em seu próprio modelo (`Produto`) e serviço (`ProdutoService`), sem acoplamento com as classes de segurança.

6.2 Separação entre validação de formulário e regra de negócio

A validação sintática dos campos (formato de e-mail, tamanho de senha, campos obrigatórios, valores numéricos não negativos) fica nas anotações de *Bean Validation* dos formulários, enquanto regras que dependem do estado do banco de dados (usuário ou e-mail já cadastrados, proteção contra remoção do último administrador) ficam na camada `service`. Essa separação mantém cada camada responsável por um tipo de regra.

6.3 Defesa em profundidade

A autorização por perfil é verificada tanto na camada de rotas HTTP (`SecurityConfig`) quanto na camada de serviço (`@PreAuthorize`), para que uma eventual falha de configuração em uma das camadas não seja suficiente para burlar o controle de acesso.

6.4 Sessão persistida no MongoDB

Em vez de manter as sessões apenas em memória, o projeto usa `spring-session-data-mongodb` para persistir as sessões autenticadas no mesmo cluster Atlas usado para os usuários e produtos, atendendo ao requisito de armazenar também as sessões no banco de dados.

7 CONSIDERAÇÕES FINAIS

O ConstruCerta atende aos requisitos funcionais e de segurança estabelecidos pela atividade: cadastro, login e logout seguros, com hash de senha e validação de dados; controle de acesso por perfil, com três papéis distintos; uso do Spring Security como mecanismo central de autenticação e autorização; persistência de usuários e sessões no MongoDB Atlas; e uma camada de apresentação em Thymeleaf desacoplada da lógica de negócio e preparada para temas visuais configuráveis. A estrutura modular do projeto também viabiliza sua reutilização futura como base de autenticação para outros sistemas.

REFERÊNCIAS

SPRING. *Spring Security Reference Documentation*. Disponível em:
<https://docs.spring.io/spring-security/reference/>. Acesso em: 8 out. 2026.

SPRING. *Spring Data MongoDB Reference Documentation*. Disponível em:
<https://docs.spring.io/spring-data/mongodb/reference/>. Acesso em: 8 out. 2026.

MONGODB, INC. *MongoDB Atlas Documentation*. Disponível em:
<https://www.mongodb.com/docs/atlas/>. Acesso em: 8 out. 2026.

THYMELEAF. *Thymeleaf Documentation*. Disponível em:
<https://www.thymeleaf.org/documentation.html>. Acesso em: 8 out. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *NBR 14724: informação e documentação – trabalhos acadêmicos – apresentação*. Rio de Janeiro: ABNT, 2011.